

# Enoncé du TP N°1

## Analyse lexicale

### Orientation

- Utiliser l'outil FLex.
- Il faut écrire des expressions régulières pour les termes spécifiés.

### Travail à faire

1. Mettez en évidence la structure générale d'un fichier Lex
2. Ecrire un analyseur (appelé MLEX) en Flex qui reconnaît les termes suivants :

**Integer** : une constante entière

**Real** : une constante réelle

**Ari\_op** : un opérateur arithmétique tel que +, -, \* et /

**Log\_op** : un opérateur logique tel que &&, || et !

**Comp\_op** : un opérateur relationnel tel que <, >, <=, >=, == et !=

**Affect\_op** : l'opérateur d'affectation =

**String** : une chaîne de caractère mise entre guillemets anglais

**Ident** : un identificateur obéissant la règle de définition des identificateurs en C

**Comment** : un commentaire délimité par /\* et \*/

**Type** : un type primitif comme int, float, double et char.

**Keyword** : un mot clé réservé comme if, else, for, break, continue, while, do, return, ...

**Separator** : un séparateur comme ( ) { } [ ] , ;

MLEX devra :

- ignorer les excès d'espaces et les sauts de lignes,
- affiche pour chaque terme identifié son unité lexicale (Type et Valeur) avec le numéro de sa ligne.

*Indication :*

Si on soumet le fichier Test.m à l'analyseur MLEX, on obtient le résultat suivant :

**Fichier Test.m :**

```
"Analyse lexicale par FLex"
```

```
float x=10.34 ;
```

```
int y=234 ;
```

```
if(y<10 && x>1.0) return 10;
```

...

### Exécution :

|        |                             |
|--------|-----------------------------|
| String | "Analyse lexicale par FLex" |
|--------|-----------------------------|

|      |       |
|------|-------|
| Type | float |
|------|-------|

|       |   |
|-------|---|
| Ident | x |
|-------|---|

|        |   |
|--------|---|
| Affect | = |
|--------|---|

|      |       |
|------|-------|
| Real | 10.34 |
|------|-------|

|           |   |
|-----------|---|
| Separator | ; |
|-----------|---|

|      |     |
|------|-----|
| Type | int |
|------|-----|

|       |   |
|-------|---|
| Ident | y |
|-------|---|

|        |   |
|--------|---|
| Affect | = |
|--------|---|

|         |     |
|---------|-----|
| Integer | 234 |
|---------|-----|

|           |   |
|-----------|---|
| Separator | ; |
|-----------|---|

|         |    |
|---------|----|
| KeyWord | if |
|---------|----|

|           |   |
|-----------|---|
| Separator | ( |
|-----------|---|

|       |   |
|-------|---|
| Ident | y |
|-------|---|

|         |   |
|---------|---|
| Comp-op | < |
|---------|---|

|         |    |
|---------|----|
| Integer | 10 |
|---------|----|

|        |    |
|--------|----|
| Log-op | && |
|--------|----|

|       |   |
|-------|---|
| Ident | x |
|-------|---|

|         |   |
|---------|---|
| Comp-Op | > |
|---------|---|

|      |     |
|------|-----|
| Real | 1.0 |
|------|-----|

|           |   |
|-----------|---|
| Separator | ) |
|-----------|---|

|         |        |
|---------|--------|
| KeyWord | return |
|---------|--------|

|         |    |
|---------|----|
| Integer | 10 |
|---------|----|

|           |   |
|-----------|---|
| Separator | ; |
|-----------|---|

...

Pour l'exécution, on utilise la commande ***flex fichier.lex***. Un fichier **lex.yy.c** est alors généré. Puis, on fait appel au service de gcc : **gcc -o prog lex.yy.c**

3. Java dispose d'une classe utilitaire nommée **StringTokenizer** (du paquetage java.util ) qui permet de découper une chaîne de caractères en différents tokens en cherchant des caractères séparateurs spécifiques. En voici un exemple d'utilisation :

#### **Testtoken.java**

```
import java.io.* ;
import java.util.* ;

public class Testtoken

{ public static void main (String args[]) throws IOException

    { String fich="Testtoken.txt";

      String mot;int i=0;

      BufferedReader entree = new BufferedReader (new FileReader (fich)) ;

      System.out.println ("Liste des mots contenus dans le fichier "+fich) ;

      while (true)

        { String ligne = entree.readLine() ;

          if (ligne == null) break ;

          StringTokenizer st = new StringTokenizer (ligne, " ") ;

          mot = st.nextToken();

          while(mot !=null) {

            i++;

            System.out.println ("Mot "+i+" : "+mot);

            mot = st.nextToken();

          }

        }

      entree.close () ;

      System.out.println ("Fin de liste ...");

    }

}
```

### **Testtoken.txt**

An interpreter is another common kind of language processor. Instead of producing a target program as a translation, an interpreter appears to directly execute the operations specified in the source program on inputs supplied by the user.

Faites l'exécution et observez les résultats.

Explorez cette classe et déduisez comment peut-on l'exploiter pour faire de l'analyse lexicale.

Faire la même chose avec python.